

Final Project

Code Comments

- **Module Header:** This module loads the Obesity dataset, clusters individuals using KMeans based on behavioral features, and identifies representative individuals per cluster.
- **Function Comments**
 - What it does: Coordinates loading, preprocessing, clustering, and analysis of the obesity dataset.
 - Inputs/Outputs:
 - Input: ObesityDataSet_raw_and_data_synthetic.csv
 - Output: cluster distribution, average cluster features, representative individuals, cluster-to-label comparison
 - High-level logic:
 - Read CSV using `csv::ReaderBuilder`
 - Extract selected numeric features into an `Array2<f64>`
 - Run `KMeans::params(k).fit()`
 - Predict cluster assignments
 - Summarize each cluster and find representative
- **In-Function Annotations**
 - At each “crucial” step (e.g. data transformations, core calculations, complicated use of iterators that cannot be easily read, etc), briefly explain why it’s there or what it does.
- **Structs & Enums**
 - One-sentence description of what the type represents and how it’s used.

2. Written Report

A. Project Overview

- **Goal:** What question are you answering?

Question:

Can we represent the obesity dataset as a graph where each node is an individual, and edges reflect lifestyle similarity, then partition this graph into k distinct behavioral groups and select k representative individuals? How well do these representatives capture the diversity of health-related behaviors, and do they align with our prior beliefs about obesity levels?

Hypothesis:

If individuals are grouped based on numerical health and lifestyle features (e.g., activity, diet, hydration, screen time), then clustering this graph will reveal distinct behavioral types. The top- k representatives—selected as the closest individuals to each cluster’s centroid—will meaningfully represent common health profiles (e.g., highly active + lean, sedentary + overweight), and likely correspond to major obesity categories such as Normal Weight, Overweight, and Obesity Types I–III.

- **Dataset:**
<https://archive.ics.uci.edu/dataset/544/estimation+of+obesity+levels+based+on+eating+habits+and+physical+condition>

B. Data Processing

- How you **loaded** it into Rust.
 - Used `csv::ReaderBuilder` to read the dataset line by line
 - Extracted relevant numeric features (Age, Weight, etc.)

Transformations:

- Converted into `ndarray::Array2<f64>` for clustering
- Labels were kept separately for post-analysis comparison

Methodology:

The dataset was clustered using Kmeans, based on 8 numeric features that describe individual health and lifestyle behaviour

1. Age
 2. Height
 3. Weight
 4. FCVC (Frequency of vegetable consumption (1-3 scale))
 5. NCP (Number of main meals per day)
 6. CH2O (Water consumption in liters per day)
 7. FAF (Physical activity frequency in hours per week)
 8. TUE (Daily screen time in hours per day)
- No **cleaning** to the csv applied.
 - These features were eliminated since they were not originally quantifiable:
 - Family history with overweight
 - CAEC (Consumption of food between meals)
 - Smoking
 - SCC (Calorie consumption monitoring)
 - CALC (Alcohol consumption)
 - MTRANS (Transportation Method Taken)
 - NObeyesdad was used at the end to for testing the clusters

C. Code Structure

1. **Modules**
 - Single file: `main.rs`
 - Handles loading, clustering, evaluation in one logical flow
2. **Key Functions & Types (Structs, Enums, Traits, etc)**
 - Purpose: full pipeline runner
 - Inputs: obesity CSV
 - Outputs: stdout print of clusters and metrics
 - Core logic: reading → parsing → clustering → summarizing
3. **For each non-trivial item, restate:**

1. CSV Parsing and Data Prep

- **Purpose:** To extract relevant numeric features and labels from the dataset for clustering.
- **Inputs/Outputs:**
 - Input: CSV file: ObesityDataSet_raw_and_data_synthetic.csv
 - Output:
 - Array2<f64>: 2D array of numeric features
 - Vec<String>: Labels (Of the level of obesity) for reference
 - Row_count: number of valid rows parsed
- **Core logic and key components:**
 - I used csv::ReaderBuilder to parse rows
 - Filters and converts selected feature columns to f64
 - Used .position() on headers to find index of each feature

2. DatasetBase Construction

- **Purpose:** Wraps the feature matrix in a structure compatible with linfa for it to work
- **Inputs/Outputs:**
 - Input: Array2<f64> from the previous step
 - Output: DatasetBase
- **Core logic and key components:**
 - Let dataset = DatasetBase::from(array.clone());

3. KMeans Model

- **Purpose:** Identifies clusters of individuals based on similarity in health/lifestyle features
- **Inputs/Outputs:**
 - Input:
 - DatasetBase objects with features
 - Number of clusters (which I decided as 3 for this project)
 - Output:
 - Trained KMeans model
- **Core logic and key components:**
 - KMeans::params(k).fit(&dataset)?
 - .centroids()

4. Predict Cluster Assignments

- **Purpose:** Assigns each data point to its closest centroid
- **Inputs/Outputs:**
 - Input:
 - Trained KMeans model
 - Original dataset
 - Output: Cluster index per row
- **Core logic and key components:**
 - model.predict(&dataset)
 - Each row is placed to a cluster (0, 1, or 2)

5. Averaging the 8 Features Per Cluster

- **Purpose:** Summarizes the average behaviours/ features of individuals in each cluster

- **Inputs/Outputs:**
 - Input: Cluster assignments
 - Output: Printed means per feature per cluster
- **Core logic and key components:**
 - Sum and average values per feature

6. Distance to Centroid Calculation

- **Purpose:** Assess the compactness of each cluster by measuring how far it is from its centroid (Euclidean distance)
- **Inputs/Outputs:**
 - Input:
 - Cluster assignments
 - Centroids
 - Output:
 - Distance by cluster
 - mean , min, max distance to be printed
- **Core logic and key components:**
 - Distances which are grouped by cluster

7. Matching the Representative Per Cluster

- **Purpose:** To identify the individual whose behaviour is most typical of the cluster (closest to centroid)
- **Inputs/Outputs:**
 - Input:
 - Cluster assignments
 - Centroids
 - Output:
 - Index of most representative person per cluster
 - Original label
- **Core logic and key components:**
 - Save index of the closest point

4. Main Workflow (In Order)

1. Load the CSV (Obesity Data Set)
2. Parse + Select Numeric Function (extracting the 8 numerical features)
3. Create Array2<f64> + wrap in dataset (each row = 1 person)
4. Wrap into Dataset Base (linfa)
5. Run K means (k =3)
6. Predict Cluster Assignments (0,1, or 2 for each person)
7. Compute pre-cluster stats (size of each, average feature values, and distance to centroids)
8. Find representative individual (closest to centroid per cluster)
9. Print result to terminal

D. Tests

- Testing was conducted **manually** by:
 - Visually verifying clustering results
 - Confirming cluster distributions and means align with expectations

- Comparing unsupervised cluster results to known obesity levels (NObeyesdad) as an added check

E. Results

- Similar lifestyle and health metrics were placed in the same cluster.
- 3 Clusters: Cluster 0, Cluster 1, and Cluster 2

```
Cluster distribution:
Cluster 0: 655 rows
Cluster 1: 707 rows
Cluster 2: 749 rows
```

```
Cluster Distance Stats (to centroid):
Cluster 0 → Avg Distance: 8.8404, Min: 1.0760, Max: 30.8851
Cluster 1 → Avg Distance: 11.1530, Min: 1.0481, Max: 56.3547
Cluster 2 → Avg Distance: 9.8200, Min: 1.3485, Max: 39.0658
```

Average Feature Values per Cluster:

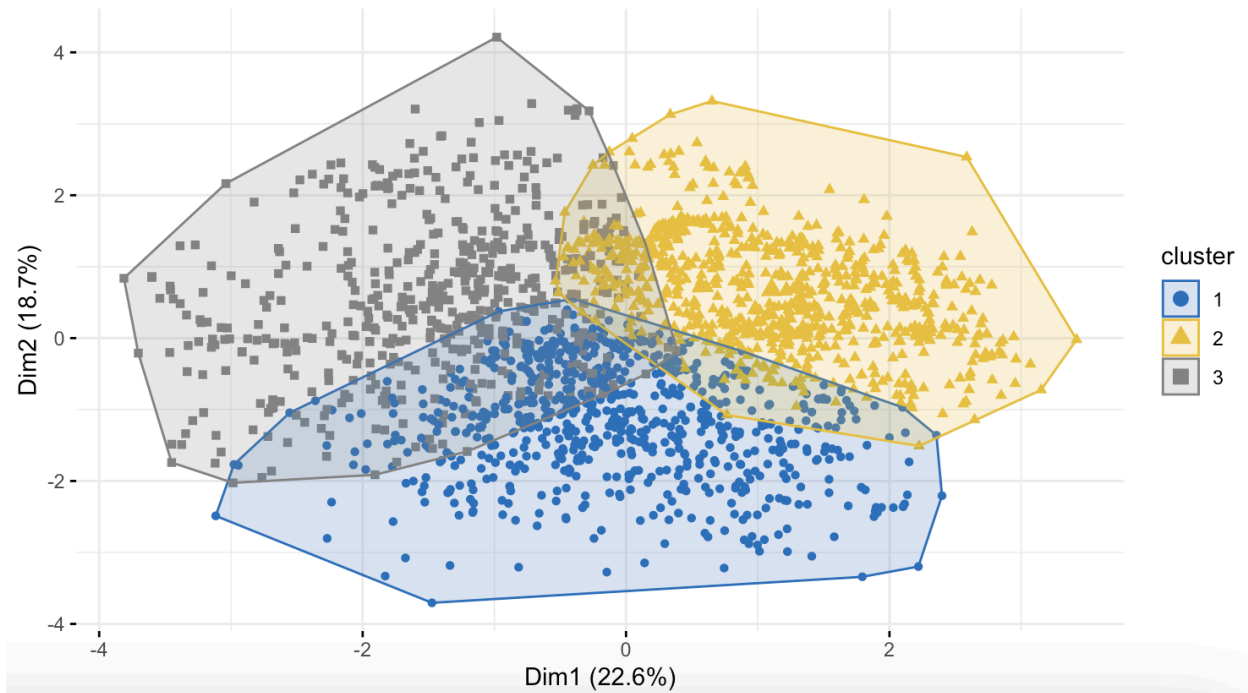
```
Cluster 0:
Age      = 20.92
Height   = 1.65
Weight   = 56.79
FCVC     = 2.36
NCP      = 2.68
CH20     = 1.90
FAF      = 1.11
TUE      = 0.75
```

```
Cluster 1:
Age      = 25.42
Height   = 1.74
Weight   = 117.15
FCVC     = 2.64
NCP      = 2.85
CH20     = 2.12
FAF      = 0.85
TUE      = 0.59
```

```
Cluster 2:
Age      = 26.23
Height   = 1.71
Weight   = 83.79
FCVC     = 2.26
NCP      = 2.54
CH20     = 2.00
FAF      = 1.08
TUE      = 0.64
```

```
★ Representative Individuals per Cluster:
Cluster 0: Representative Index = 64, Label = Normal_Weight
Cluster 1: Representative Index = 1737, Label = Obesity_Type_II
Cluster 2: Representative Index = 944, Label = Overweight_Level_I
```

K-means Cluster estimate data visualization using R



*Important Note: Cluster assignment order and centroids may differ slightly because Rstudio and Rust may have different centroid initializations.

Interpretation:

- Cluster 0: Mostly underweight/normal-weight individuals
- Cluster 1: Obese individuals with the highest average weight
- Cluster 2: Transitional — many overweight or Obesity Type I

The clustering matched known labels despite being unsupervised, confirming that health and lifestyle features are strongly predictive of obesity categories.

Cluster Composition

- Clusters are relatively balanced, none of them are disproportionately large or small, which suggests natural separability in the dataset

Cluster 0: 655 people (31%)

Cluster 1: 707 people (33%)

Cluster 2: 749 people (36%)

Distance to Centroid

- Cluster one has the highest average distance and max distances, indicating that people part of this cluster have more “extreme” or inconsistent lifestyle metrics. Moreover, this also indicates the possibly higher variance in how obesity manifests in a person

Behavioural and Health Profile per Cluster

- Cluster 0: This is the youngest group with the lightest weight. They have the most physical activity, but also have the highest screen time.
- Cluster 1: Extremely high weight compared to the other clusters. They also surprisingly consume higher amounts of vegetables. They also have the lowest physical activity.
- Cluster 2: People in the cluster are usually the in-between of cluster 0 and 1. They are slightly more active than cluster 1, but do not have as healthy of a lifestyle as compared to cluster 0.

Thus, the clusters reveal that behavioural separations align closely with obesity labels, even though it was excluded when doing the Kmeans clustering test. Thus, representative individuals make intuitive sense.

F. Usage Instructions

- How to build and run your code.
- Description of any command-line arguments or user interaction in the terminal
 - Cargo build for the added dependencies
 - Cargo run
 - Output is printed directly to terminal
- Expected Run Time: Less than 10 seconds (Should be relatively quick)

G. Sources and AI-Assistance Disclosure and Other Citations

- Chat GPT use:
 - Clarify use of linfa, ndarray, and KMeans in Rust
 - What I learned: I learned the application of these, and how to implement them into my rust code. I also learned how Linfa is similar to Python scikit-learn. I was able to apply this to my rust code.

Absolutely — here's a clear explanation of how `linfa`, `ndarray`, and `KMeans` work together in your Rust project:

✓ 1. `linfa`: The Machine Learning Framework

`linfa` is a Rust machine learning library — similar to Python's `scikit-learn`.

🔧 Why You Use It:

- Provides the `KMeans` algorithm via `linfa-clustering`
- Supplies the `DatasetBase` struct to wrap feature arrays
- Offers the `Fit` and `Predict` traits used in your `.fit()` and `.predict()` calls

🧠 Example:

```
rust                                                                    Copy Edit

let model = KMeans::params(3).fit(&dataset?);
let cluster_assignments = model.predict(&dataset);
```

- Example:
 - How to implement the cluster distance equation to the code
 - What I learned: How to implement computing the euclidean distance to Rust codes

🧩 Step-by-Step Implementation

📌 Step 1: After fitting the KMeans model, get the centroids

Right after fitting the model, store the centroids:

```
rust                                                                    Copy Edit

let model = KMeans::params(n_clusters).fit(&dataset?);
let centroids = model.centroids(); // <-- Required for distance calculation
let cluster_assignments = model.predict(&dataset);
```

📌 Step 2: Prepare a data structure to store distances per cluster

Create a vector of vectors to store distances for each cluster:

```
rust                                                                    Copy Edit

let mut distances_by_cluster = vec![Vec::new(); n_clusters];
```


🧠 Step 3: Loop through each row and compute the distance to its cluster's centroid

```
rust                                                                    Copy Edit

for (i, row) in array.outer_iter().enumerate() {
    let cluster = cluster_assignments[i]; // which cluster this row belong
    let centroid = centroids.row(cluster); // get the corresponding centro

    let distance = row
        .iter()
        .zip(centroid.iter())
        .map(|(a, b)| (a - b).powi(2))
        .sum::<f64>()
        .sqrt(); // Euclidean distance

    distances_by_cluster[cluster].push(distance); // store it
}
```

- Other sources:
 - What is K-means clustering: <https://www.ibm.com/think/topics/k-means-clustering>
 - Narray usage in rust: <https://docs.rs/ndarray/latest/ndarray/#:~:text=The%20ndarray%20crate%20provides%20an,by%20using%20that%20many%20indices.>
 - Kmeans in Rust: <https://docs.rs/kmeans/latest/kmeans/>
 - Euclidean Distance: <https://www.geeksforgeeks.org/euclidean-distance/>
 - Got help from tutor